

Спецификация кастомной мобильной ОС

на базе форка Fuchsia / Zircon · целевое железо Google Pixel 9 (Tensor G4)

Полная инженерная проработка по слоям · карта (пере)использования · роадмап и риски

Статусы модулей: как есть · обернуть · порт · реверс · форк · с нуля

О документе. Это инженерная карта решений, собранная из проработки всех аспектов: аппаратной платформы, ядра, GPU, телефонии, камеры, совместимости и продуктового слоя, плюс сквозные аспекты, дорожная карта, оценки трудозатрат, риски и правовая рамка реверс-инжиниринга. Документ честен по потолкам (камера и телефония имеют известные ограничения) и по объёму (аппаратная часть — сложнее Asahi). Ключевая стратегия — развязка двух треков: продукт строится в эмуляторе уже сейчас, железо поднимается отдельно. Не является гарантией осуществимости.

Содержание

1. Введение и видение

- 1.1 Резюме
- 1.2 Цели и не-цели
- 1.3 Видение: ОС вокруг сущностей и агентов
- 1.4 Обоснование выбора железа: Pixel 9 / Tensor G4
- 1.5 Почему Fuchsia/Zircon — и честные минусы
- 1.6 Принципы и ограничения проекта
- 1.7 Глоссарий

2. Стратегия и методология

- 2.1 Стратегия двух треков
- 2.2 Таксономия (пере)использования
- 2.3 Развязка: substrate / platform / shell
- 2.4 Оценка трудозатрат и сравнение с ориентирами
- 2.5 Реестр рисков (обзор)
- 2.6 Правовая рамка реверс-инжиниринга

3. Аппаратная платформа (Tensor G4 / Pixel 9)

- 3.1 Обзор SoC Tensor G4
- 3.2 Карта памяти и device tree
- 3.3 Цепочка загрузки и загрузчик
- 3.4 Bring-up ядра SoC
- 3.5 Дисплейная подсистема
- 3.6 Тач и сенсоры
- 3.7 Питание: PMIC, зарядка, энергопотребление
- 3.8 Хранилище (UFS)
- 3.9 Связь: Wi-Fi, BT, NFC, GNSS
- 3.10 Titan M2 и доверенная загрузка

4. Ядро и субстрат ОС (Fuchsia / Zircon)

- 4.1 Архитектура Zircon
- 4.2 Капабилити-модель и хэндлы
- 4.3 Driver Framework (DFv2)
- 4.4 Компонентный фреймворк и FIDL
- 4.5 Магма — графический субстрат
- 4.6 Стратегия форка дерева Fuchsia
- 4.7 Система сборки (GN/ninja, fx)
- 4.8 Эмулятор FEMU и рабочий цикл разработки

5. GPU и графика

- 5.1 Архитектура Mali-G715 (Valhall, CSF)
- 5.2 Экосистема открытых драйверов Mali
- 5.3 Магма и портирование драйвера
- 5.4 Методология реверс-инжиниринга GPU
- 5.5 Прошивка GPU и блобы
- 5.6 Композитор и вывод на экран

6. Телефония и модем (Exynos 5400)

- 6.1 Обзор Exynos Modem 5400

- 6.2 Стек телефонии целиком
- 6.3 Link-транспорт (SIPC)
- 6.4 Boot-секвенс модема
- 6.5 IPC-протокол (libsamsung-ipc)
- 6.6 RE набора команд (BaseMirror)
- 6.7 RIL / ModemManager / ofono
- 6.8 Голосовой тракт
- 6.9 SMS, данные, регистрация, экстренные вызовы
- 6.10 Регуляторные аспекты

7. Камера и вычислительная фотография

- 7.1 Камерное железо
- 7.2 Конвейер захвата и обработки
- 7.3 libcamera и software-ISP
- 7.4 HDR+ пайплайн
- 7.5 Тюнинг под сенсор
- 7.6 Нейросетевые/learned подходы
- 7.7 Анализ потолка качества

8. Совместимость с Linux/Android (Starnix)

- 8.1 Архитектура Starnix
- 8.2 Запуск Linux/bionic-бинарей
- 8.3 binder в Starnix
- 8.4 Где Starnix не помогает

9. Userland / продуктовый слой

- 9.1 Архитектура продукта
- 9.2 Shell и композитор (Scenic/Flatland)
- 9.3 Модель entity/agent
- 9.4 Глобальная история
- 9.5 Local-first данные и CRDT-синк
- 9.6 Интеграции, датасорсы, виджеты
- 9.7 Apple-интероп (AirDrop/AirPlay/буфер)
- 9.8 UX-парадигма

10. Сквозные аспекты

- 10.1 Модель безопасности
- 10.2 Обновления и OTA
- 10.3 Приватность
- 10.4 Производительность и энергопотребление
- 10.5 Наблюдаемость и отладка
- 10.6 Стратегия тестирования
- 10.7 Сборка, CI и воспроизводимость

11. Дорожная карта и исполнение

- 11.1 Рoadmap (фазы)
- 11.2 Трек A: вехи (FEMU)
- 11.3 Трек B: вехи (железо)
- 11.4 Оценки трудозатрат по модулям
- 11.5 Критический путь и зависимости
- 11.6 Распределение по статусам
- 11.7 Минимальные жизнеспособные конфигурации

12. Сравнение базовых платформ

- 12.1 postmarketOS (mainline Linux)
- 12.2 Halium / Droidian
- 12.3 Genode / Sculpt (микроядро на телефоне сегодня)
- 12.4 Сводное сравнение
- 12.5 Почему всё же Fuchsia и когда пересмотреть

13. Детализация bring-up драйверов (L1)

- 13.1 UART и ранний консоль
- 13.2 GIC и прерывания
- 13.3 Тактирование (clocks) и pinctrl
- 13.4 PMIC и регуляторы
- 13.5 Suspend/resume и энергопотребление
- 13.6 Дисплей: DSI, панель, инициализация
- 13.7 UFS подробнее
- 13.8 Wi-Fi / BT: чип, firmware, стек

14. Контракты слоёв (FIDL)

- 14.1 Принцип контрактов
- 14.2 Контракт substrate → platform
- 14.3 Контракт platform → shell
- 14.4 Mock-substrate для FEMU

15. Модель данных entity/agent

- 15.1 Схема сущностей и рёбер
- 15.2 Жизненный цикл агента
- 15.3 Запросы и индексация
- 15.4 Слияние и дедупликация
- 15.5 CRDT и синхронизация подробнее

16. Модем: глубокое погружение

- 16.1 Формат кадра SIP
- 16.2 Ландшафт протоколов: QMI / MBIM / AT — и почему Samsung свой
- 16.3 ModemManager / ofono и плагины
- 16.4 Состояния звонка и маршрутизация
- 16.5 Тестовый путь телефонии

17. Камера: глубокое погружение

- 17.1 Стадии ISP-конвейера
- 17.2 Цветовая наука и тюнинг
- 17.3 Raw / DNG
- 17.4 Выравнивание и слияние бурста (HDR+)
- 17.5 Видео-путь

18. Среда разработки, сборка и CI

- 18.1 Машина и зависимости
- 18.2 Сборка Fuchsia (GN/ninja/fx)
- 18.3 FEMU и отладка
- 18.4 CI и тестовые контуры
- 18.5 Управление форком и upstream

19. Уроки прецедентов

- 19.1 Asahi Linux
- 19.2 Replicant / libsamsung-ipc
- 19.3 dahliaOS
- 19.4 postmarketOS / Megapixels
- 19.5 Genode / Sculpt
- 19.6 Синтез уроков

20. Приложения

- 20.1 А. Полная инвентаризация модулей
- 20.2 В. Расширенный реестр рисков
- 20.3 С. Ключевые проекты и ссылки
- 20.4 D. Открытые вопросы
- 20.5 Е. Легенда статусов (справочно)

Карта системы: (пере)использование по слоям

Кастомная мобильная ОС на Fuchsia / Zircon — карта (пере)использования

Цель железа: Pixel 9 (Tensor G4 · Mali-G715 · Exynos 5400). Слои снизу вверх; цвет = статус работы над модулем.

L6 · Userland / Продукт

наше видение ОС

Shell + композитор
(на Scenic / Flatland)

С НУЛЯ

Entity / Agent
фреймворк

С НУЛЯ

Глобальная история
всей ОС

С НУЛЯ

Local-first данные
+ синк (CRDT)

С НУЛЯ

Интеграции / датасорсы
/ виджеты

С НУЛЯ

Apple-интероп
OpenDrop · uxplay · KDE Connect

ОБЕРНУТЬ

L5 · Совместимость

мост в мир Linux / Android-userspace

Starnix — Linux UAPI + binder поверх Zircon · гоняет libsamsung-ipc, libcamera, hdr-plus, KDE Connect как Linux-бинари

КАК ЕСТЬ

L4 · Камера / выч. фотография достойно, но не Pixel-grade

HDR+ пайплайн hdr-plus + Halide	libcamera userspace + software-ISP	Сенсор + ISP capture-драйвер
КАК ЕСТЬ	ОБЕРНУТЬ	РЕВЕРС
	Тюнинг сенсора CCM · шум · AWB	
	С НУЛЯ	

L3 · Телефония модем — чёрный ящик, которым командуем

RIL / ModemManager логика	librsmsm-ipc IPC-протокол	Команды Exynos 5400 (BaseMirror RE)
ОБЕРНУТЬ	ПОРТ	РЕВЕРС
Modem link-транспорт (SIPC, /dev/umts_ipc0)	Boot-секвенс модема	Прошивка модема (подписанный blob)
ПОРТ	РЕВЕРС	КАК ЕСТЬ

L2 · GPU

Mali-G715 (Valhall)

Мagma
 граф. стек Fuchsia

КАК ЕСТЬ

Мali-драйвер на Magma
 (знания Panfrost/Panther)

ПОРТ

GPU firmware
 (blob)

КАК ЕСТЬ

L1 · Bring-up Tensor G4 недокументированный SoC

- Ядро SoC: CPU · GIC таймеры · UART · clocks** (PEBEP3)
- Дисплей-контроллер** (PEBEP3)
- Тач / сенсоры** (ПОРТ)
- USB / PMIC / зарядка** (PEBEP3)
- Загрузчик / ZBI-цепочка** (PEBEP3)

LO · Ядро / микроядро

капабилити-фундамент

Дерево Fuchsia
(наш форк)

ФОРК

Zircon
микроядро

КАК ЕСТЬ

Driver Framework
(DFv2)

КАК ЕСТЬ

КАК ЕСТЬ берём без изменений	ОБЕРНУТЬ запускаем/оборачиваем существующее
ПОРТ переносим код/знания на нашу платформу	РЕВЕРС реверс-инжиниринг с нуля
ФОРК форкаем и ведём свою ветку	С НУЛЯ пишем новый код

Введение и видение

Зачем проект существует, что именно строим, на каком железе и почему — и где границы амбиций.

1.1 · Резюме

Документ описывает полную спецификацию кастомной мобильной операционной системы, построенной на форке Fuchsia с микроядром Zircon, с собственным userland в парадигме «сущности и агенты» (entity/agent-first), и нацеленной на железо Google Pixel 9 (SoC Tensor G4). Цель — не очередной Android-форк и не дистрибутив, а самостоятельная ОС, где организующей единицей являются не файлы и приложения, а проекты, задачи, документы, люди, места и события, связываемые фоновыми агентами, с глобальной историей по всей системе, local-first данными, прозрачной синхронизацией между устройствами и интеграцией с экосистемой Apple/Linux.

Главный вывод всей проработки: проект распадается на две принципиально разные по сложности и риску части, которые следует развязать. Продуктовый слой (оболочка, модель сущностей/агентов, история, синхронизация) полностью OS-независим и может разрабатываться и запускаться в эмуляторе FEMU уже сейчас, без единого аппаратного блокиера. Аппаратный bring-up Pixel 9 (поднять Zircon на недокументированном Tensor, написать драйверы GPU/модема/камеры, частично через реверс-инжиниринг) — отдельная, многолетняя, по объёму превосходящая Asahi Linux работа, в которой модем и камера дают рабочий, но не флагманский результат.

Одно предложение. Форкаем дерево Fuchsia; Zircon/DFv2/Magma/Starnix берём как есть; Linux-форменные библиотеки оборачиваем через Starnix; драйверы GPU и транспорта модема портируем из RE-знаний; bring-up Tensor и протокол модема реверсим; весь продуктовый слой пишем с нуля.

1.2 · Цели и не-цели

Цели

- Самостоятельная ОС с микроядром и капабилити-моделью безопасности.
- Userland, организованный вокруг сущностей (проект/задача/документ/человек/место/агент), а не приложений.
- Глобальная история всей системы — как история браузера, но для ОС целиком.
- Local-first данные с опциональной синхронизацией между устройствами (CRDT).
- Подключаемые интеграции, источники данных и адаптивные виджеты.
- Интероперабельность с macOS/Ubuntu: AirDrop/AirPlay-аналоги, общий буфер обмена.
- Открытый исходный код всего, что мы пишем сами.

Не-цели (сознательно вне области)

- Паритет камеры с Pixel-on-Android (Night Sight, Super Res Zoom) — недостижим без закрытого GCam.
- Сертифицированная радиочасть с модифицированной прошивкой модема — вне закона и вне задач.
- Поддержка произвольного парка телефонов — целимся в одну модель (Pixel 9).
- Совместимость с Google Play Integrity и проприетарными AI-функциями Pixel.
- Коммерческая готовность «из коробки» — это исследовательско-инженерный проект.

1.3 · Видение: ОС вокруг сущностей и агентов

Базовая абстракция системы — не приложение и не файл, а «сущность» (entity): человек, место, событие, документ, проект, задача, сообщение, устройство. Сущности связаны рёбрами (участвует, относится к, порождён из) и живут в едином local-first хранилище. Над потоками данных работают «агенты» — фоновые компоненты, которые извлекают сущности из источников (почта, календарь, мессенджеры, файлы, датчики), связывают их и поддерживают глобальную историю действий.

Поверхность взаимодействия — не сетка иконок, а адаптивные «поверхности» (виджеты), собираемые вокруг текущей сущности или задачи, плюс глобальный поиск/история. Любопытно, что эта модель почти дословно совпадает с ранним дизайном самой Fuchsia (entities/agents, лента-история, Ledger для синхронизации между устройствами) — поэтому дизайн-документы Fuchsia служат и источником вдохновения, и предупреждением о том, что в этой модели сложно.

Практическое следствие для архитектуры: продуктовый слой — это хранилище сущностей + агенты + оболочка/композитор. Он не зависит от того, какое под ним ядро или драйверы, и именно поэтому его можно (и нужно) строить первым, в эмуляторе.

1.4 · Обоснование выбора железа: Pixel 9 / Tensor G4

Pixel 9 выбран как целевое железо по совокупности: разблокируемый загрузчик (на unlocked-моделях), эталонный статус Pixel для кастомных ОС, и GPU ARM Mali-G715 (Valhall), под который существует самая зрелая открытая экосистема драйверов (Panfrost/Panthor/PanVK), что критично для реверса графики. Это делает Pixel 9 заметно более удачной мишенью, чем Pixel 10 (где GPU сменился на Imagination DXT — без открытого драйвера) для GPU-части.

Одновременно нужно честно зафиксировать минусы именно этой мишени. Tensor G4 (кодовое имя zumapro) — кастомный SoC Samsung/Google, недокументированный публично; CPU-комплекс 1×Cortex-X4 + 3×Cortex-A720 + 4×Cortex-A520; модем внешний — Samsung Exynos Modem 5400, проприетарный и поколения 2024 года, не покрытый существующими открытыми реализациями. То есть «причины хотеть Pixel» (камера и звонки) — ровно то, что хуже всего поддаётся в кастомной ОС.

Подсистема	Pixel 9 (Tensor G4)	Последствие для проекта
CPU	1×X4 @3.1 + 3×A720 + 4×A520, Samsung 4 нм	стандартные ARM-ядра; bring-up в Zircon — ручной, но без экзотики
GPU	ARM Mali-G715 (Valhall, ~940 МГц)	есть Panfrost/Panthor → лучший открытый таргет для RE
Модем	Exynos Modem 5400 (внешний)	проприетарный, новый; протокол требует RE; голос — отдельно
NPU/ISP/IP	Edge TPU, GXP, BigWave, ISP	закрытые IP-блоки; AI/камера-полировка недоступны
Secure element	Titan M2	доверенная загрузка; relock возможен, но AI-функции отпадут
Загрузчик	разблокируемый (не carrier)	обязательно unlocked-модель; Verizon — нет

1.5 · Почему Fuchsia/Zircon — и честные минусы

Fuchsia даёт то, ради чего затевается «совсем кастомная» ОС: микроядро Zircon с капабилити-моделью, компонентную архитектуру с явными контрактами (FIDL), современную модель драйверов (DFv2), аппаратную поддержку MTE и встроенный путь совместимости с Linux/Android-userspace (Starnix). Дерево открыто (Zircon — MIT-style, userspace — BSD/Apache), что делает форк законным. Прецедент форка существует — dahliaOS.

Минусы, которые нельзя замалчивать. Во-первых, драйверная стена: Fuchsia не использует ядро Linux, поэтому ни один Linux/Android-драйвер железа не переносится «как есть» — всё пишется против DFv2/Magma. Во-вторых, крошечное внешнее сообщество: bring-up на новом ARM-SoC — почти в одиночку, без инфраструктуры вроде pmbootstrap. В-третьих, движущая мишень: API и сборка контролируются Google и меняются. В-четвёртых, Google так и не довела Fuchsia до daily-driver на телефонах — официальное железо это x86 (NUC-класс), эмулятор и узкий набор ARM-плат, а не смартфоны.

Вывод по выбору базы. Fuchsia оправдана, если микроядро и продуктовая модель важнее, чем быстро поехать на флагмане. Для «потрогать микроядро на телефоне сегодня» альтернатива — Genode/Sculpt на PinePhone. Для «открытые/RE-драйверы заработают проще» альтернатива — Linux-база (postmarketOS/Halium). Этот документ описывает путь Fuchsia осознанно, с принятием его цены.

1.6 · Принципы и ограничения проекта

- Развязка прежде всего: substrate, платформенные сервисы и оболочка — независимые слои с явными контрактами.
- Сначала ценность без железа: всё, что можно сделать в эмуляторе, делается первым.
- Переиспользование вместо переписывания: каждый модуль проходит через таксономию (как есть / обернуть / порт / реверс / форк / с нуля).
- Честность по потолкам: камера и телефония имеют известные ограничения качества — они зафиксированы, не замаскированы.
- Никаких блобов сверх необходимого, но и без фанатизма: подписанные firmware-блобы (GPU/модем) используются как есть, не реверсятся.
- Clean-room для реверса: документируем интерфейс железа, реализуем по спецификации, не копируем дизассемблированный код.

1.7 · Глоссарий

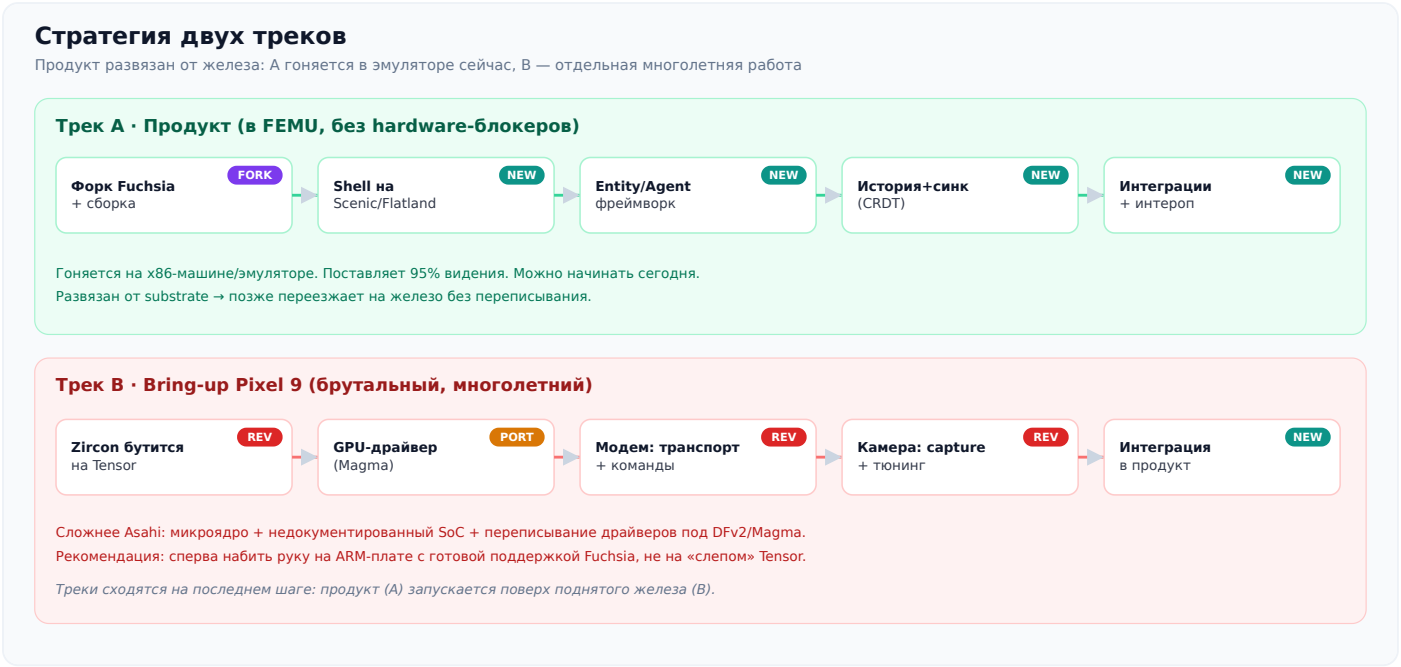
Термин	Значение
Zircon	микроядро Fuchsia (~170 syscalls, капабилити, хэндлы)
DFv2	Driver Framework v2 — модель драйверов Fuchsia
FIDL	язык описания интерфейсов для IPC между компонентами Fuchsia
Magma	графический субстрат Fuchsia (аналог DRM/Mesa-границы)
Starnix	рантайм Linux-ABI (+binder) поверх Zircon, без VM
FEMU	эмулятор Fuchsia для разработки на x86
Scenic/Flatland	система композиции/рендеринга UI во Fuchsia
SIPC	Samsung IPC — протокол канала AP↔модем
RIL	Radio Interface Layer — слой телефонии над модемом
ISP	Image Signal Processor — обработчик сигнала камеры
CCM	Color Correction Matrix — часть тюнинга камеры
CRDT	бесконфликтный реплицируемый тип данных (для синка)
MTE	Memory Tagging Extension — аппаратная защита памяти
bring-up	первичный запуск ОС на конкретном железе

Стратегия и методология

Как мы принимаем решения о (пере)использовании, как развязываем слои, как оцениваем труд и риски, и в каких правовых рамках реверсим.

2.1 · Стратегия двух треков

Центральное стратегическое решение — расцепить продукт и железо в два параллельных трека с разными профилями риска. Трек А (продукт) строится в эмуляторе и поставляет основную ценность видения без аппаратных блокировок. Трек В (железо) — длинный, рискованный bring-up Pixel 9. Треки сходятся только в самом конце, когда продукт запускается поверх поднятого железа.



Два трека: продукт в FEMU сейчас, железо — отдельно и долго

Практический смысл: даже если Трек В затянется на годы или упрётся в непреодолимый блок (например, голосовой тракт модема), Трек А остаётся работоспособным продуктом, который можно демонстрировать, тестировать и развивать на эмуляторе и на поддерживаемом Fuchsia железе.

2.2 · Таксономия (пере)использования

Каждый модуль системы классифицируется одним из шести статусов. Статус определяет и объём работы, и юридический режим, и риск.

Статус	Что означает	Когда применяем	Типичный риск
КАК ЕСТЬ	берём готовое без изменений	компонент работает под нашу задачу напрямую	низкий
ОБЕРНУТЬ	запускаем существующее в обёртке (Starnix/контейнер)	Linux-форменный userspace, который не хочется переписывать	средний (совместимость ABI)
ПОРТ	переносим код/знания на нашу платформу	есть RE-знания или код, но под другой фреймворк	высокий (объём)
РЕВЕРС	реверс-инжиниринг с нуля	закрытый интерфейс без открытой реализации	очень высокий (неопределённость)
ФОРК	форкаем и ведём свою ветку	нужна своя дивергентная версия базы	средний (сопровождение)
С НУЛЯ	пишем новый код	оригинальная функциональность	зависит от объёма

Дерево решений простое: если есть готовое и подходит — КАК ЕСТЬ; если есть, но под Linux, а мы на Fuchsia — ОБЕРНУТЬ через Starnix (для userspace) или ПОРТ (для kernel-уровня); если интерфейс закрыт и открытой реализации нет — РЕВЕРС; всю базу ОС — ФОРК; продуктовую логику — С НУЛЯ.

2.3 · Развязка: **substrate** / **platform** / **shell**

Архитектура делится на три горизонта с явными контрактами между ними, чтобы продукт не зависел от железа и базы:

- Substrate — ядро, драйверы, HAL, firmware. Самый «грязный» и железо-зависимый слой (Трек В).
- Platform services — хранилище сущностей, синхронизация, история, интеграции, телефония/камера как сервисы с FIDL-контрактами.
- Shell / UI — оболочка, композитор, поверхности, поиск.

Контракт между platform и substrate — стабильные FIDL-интерфейсы (например, «дай кадр камеры», «отправь IPC-команду модему», «отрисуй поверхность»). Тогда продукт собирается и в FEMU (где substrate эмулируется), и на железе (где substrate реальный), без изменений выше границы.

2.4 · Оценка трудозатрат и сравнение с ориентирами

Объём оценивается относительно двух исторических ориентиров. Asahi Linux (реверс Apple Silicon) — блестящая многочеловеческая команда, годы работы, относительно предсказуемая мишень, и, главное, целью был Linux, где драйверная экосистема уже есть. Replicant/libsamsung-ipc (свободная телефония на Samsung) — годы на один класс модемов, и до сих пор не покрывает современные модели.

Наш проект по аппаратной части — это «Asahi минус команда, минус Linux-экосистема, минус документация, плюс микроядро». Поэтому реалистичный горизонт Трека В — единицы лет даже при удачном RE, а отдельные блоки (голос модема, ISP-полировка) могут не закрыться вовсе. Это не повод не начинать — это повод развязать треки и не делать продукт заложником железа.

Принцип оценки. Трудозатраты в этом документе даны в условных единицах (S/M/L/XL) и фазах, а не в человеко-месяцах: на недокументированном железе разброс слишком велик, чтобы давать календарные обещания.

2.5 · Реестр рисков (обзор)

Риск	Влияние	Вероятность	Митигация
Zircon не бутится на Tensor в разумный срок	критическое для Трека В	средняя	начать с поддержанной Fuchsia ARM-платы; ранний UART
Голосовой тракт модема не заводится	высокое (нет звонков)	высокая	сначала данные/SMS; голос — отдельная веха, допускающая провал
GPU-драйвер на Мagma буксует	высокое (нет UI-ускорения)	средняя	software-рендеринг как временный путь; опереться на Panfrost-знания
Google меняет API Fuchsia/Starnix	среднее	высокая	пиннинг ревизии; абстракция над нестабильными API
RE-объём недооценён	высокое (сроки)	высокая	развязка треков; MVP-конфигурации; принятие урезаний
Юридические претензии по RE	низкое-среднее	низкая	clean-room; RE строго для интероперабельности; firmware не трогаем

2.6 · Правовая рамка реверс-инжиниринга

Реверс-инжиниринг ради интероперабельности широко разрешён: в ЕС — декомпиляция для совместимости по Директиве о ПО (Art. 6), в США — исключение DMCA §1201(f). Авторское право Сербии гармонизировано с правом ЕС. Это не юридическая консультация; нюансы зависят от юрисдикции.

Безопасная практика — clean-room: одна сторона наблюдает и документирует интерфейс железа (командный поток GPU, кадры IPC модема), пишет спецификацию; другая реализует по спецификации, не видя оригинального кода. Большинство успешных открытых драйверов (Freedreno, Panfrost) так и сделаны — реверсится поведение железа, а не копируется дизассемблированный драйвер.

- Firmware-блобы (GPU, baseband) не реверсим и не модифицируем — используем подписанные как есть.
- Радиоповедение модема не изменяем (регуляtorика) — только командуем через штатный интерфейс.
- Патенты — отдельная ось от копирайта (как у кодеков в FFmpeg); учитываем для специфичных алгоритмов.
- Документируем источники RE-знаний (например, командные форматы из Panfrost) для прозрачности происхождения.

Аппаратная платформа (Tensor G4 / Pixel 9)

Полная инвентаризация подсистем Pixel 9 и что каждая требует для bring-up под Zircon.

3.1 · Обзор SoC Tensor G4

Tensor G4 — кастомный SoC на базе Samsung Exynos-родословной, изготовленный по 4-нм процессу Samsung. Публичной документации (даташита, TRM) нет — это фундаментальное ограничение всего bring-up: регистровые карты, тактовые домены, контроллеры периферии приходится восстанавливать из downstream-ядра Pixel и наблюдением.

Блок	Детали	Статус bring-up
CPU	1xCortex-X4 @3.1, 3xA720 @2.6, 4xA520 @1.95	РЕВЕРС (init, GIC, таймеры)
GPU	ARM Mali-G715 (Valhall, CSF)	ПОРТ (Panfrost-знания → Magma)
Модем	Exynos Modem 5400 (внешний)	РЕВЕРС (протокол/boot)
TPU/NPU	Edge TPU	вне области (закрыт)
ISP	кастомный ISP	РЕВЕРС (capture) / вне области (полировка)
Secure	Titan M2	КАК ЕСТЬ (доверенная загрузка)
Память	LPDDR5	РЕВЕРС (карта памяти)
Сторадж	UFS	РЕВЕРС/ПОРТ

3.2 · Карта памяти и device tree

Первый артефакт bring-up — device tree (или его эквивалент для Zircon — board driver/board-specific конфигурация), описывающий карту памяти, базовые адреса контроллеров, прерывания, тактовые домены. Google уже выложила предварительные DT для Pixel 10 в mainline Linux (boot до текстовой консоли, с оговоркой про «ещё не выпущенный загрузчик») — для Pixel 9 ориентиром служат downstream-исходники ядра Pixel и эти патчи.

Важно: Zircon не использует Linux-DT напрямую; информация о плате подаётся через board driver и ZBI-элементы. Поэтому DT-знания из Linux транслируются в формат Zircon вручную — это ПОРТ знаний, не файла.

3.3 · Цепочка загрузки и загрузчик

Pixel загружается через цепочку, оканчивающуюся ABL (Android Bootloader, на базе Little Kernel). Чтобы загрузить Zircon, нужно либо цепочкой из ABL передать управление в Zircon ZBI-образ, либо заменить часть загрузчика. Патчи Google для mainline Linux на Pixel 10 прямо отмечают зависимость от «ещё не выпущенного загрузчика» — это сигнал, что путь загрузки нетривиален и частично закрыт.

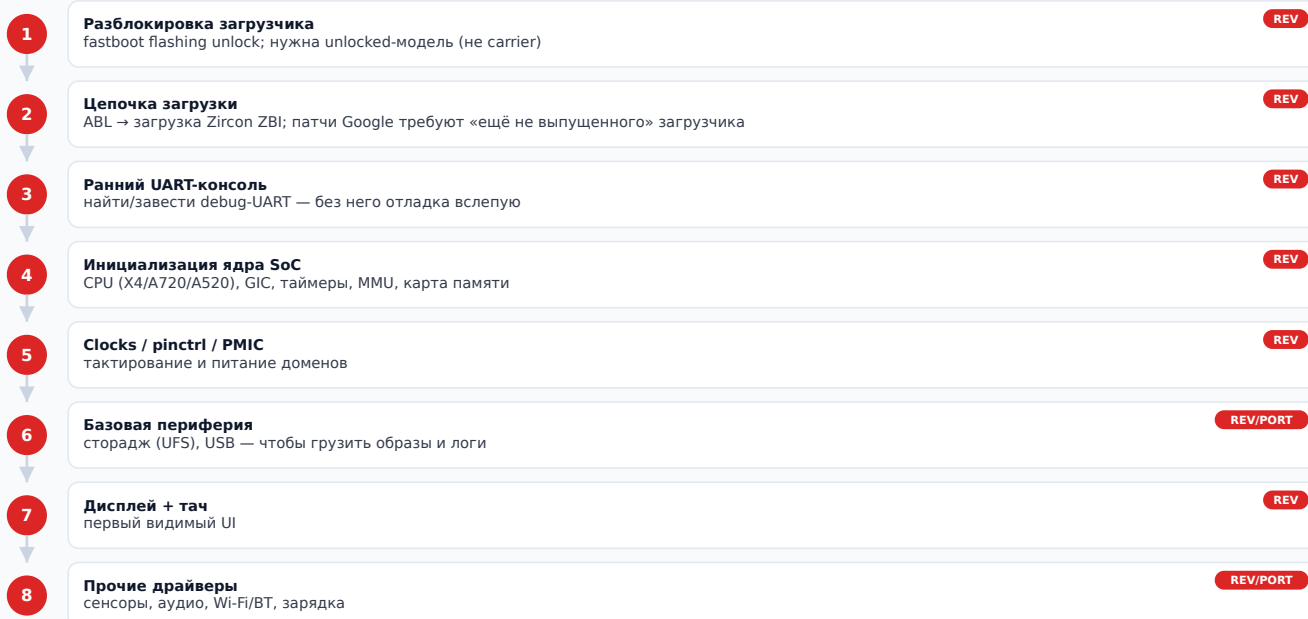
- Разблокировать загрузчик (только unlocked-модель; OEM unlocking; стирание данных).
- Завести ранний UART-консоль — без него отладка bring-up почти невозможна.
- Подготовить путь A/B + verified boot или временно отключить verified boot на этапе разработки.
- Загрузить Zircon ZBI; на финальном этапе — relock со своим ключом (ценой потери AI-функций).

3.4 · Bring-up ядра SoC

После загрузки Zircon нужно завести SoC на минимальном уровне: инициализация CPU-ядер и кэшей, GIC (контроллер прерываний), системные таймеры, MMU и карта памяти, тактирование (clocks), pinctrl, PMIC. Это та же стена, об которую бьётся mainline Linux на Tensor — даже Tensor G1 дошёл до «busybox shell» лишь спустя ~2.5 года. Здесь нет наследия Exynos-драйверов Linux, на которое можно опереться напрямую (мы на Zircon), поэтому объём — максимальный.

Bring-up Tensor G4: путь до загрузки Zircon

Каждый шаг — отдельная RE-задача; даташита нет, ориентир — downstream-ядро Pixel



Последовательность bring-up: каждый шаг — отдельная RE-задача

3.5 · Дисплейная подсистема

Дисплей — первый видимый результат и потому ранний приоритет после консоли. Нужны драйвер дисплей-контроллера (DSI/панель), инициализация панели (часто Samsung AMOLED со своей последовательностью команд), и интеграция с композитором Fuchsia (Scenic/Flatland) через display-coordinator. Источник знаний — downstream-драйвер панели; реализация — нативный Fuchsia display driver (PEBEP/ПОРТ).

3.6 · Так и сенсоры

Такскрин обычно подключён по I2C/SPI с относительно стандартным контроллером — один из самых посильных блоков (ПОРТ, частично PEBEP). Сенсоры (акселерометр, гироскоп, магнитометр, освещённость, приближение) — набор I2C-устройств; часть имеет стандартные драйверы-аналоги. Сканер отпечатка (ультразвук под экраном) и Face Unlock — отдельный, более закрытый блок, выносимый за пределы MVP.

3.7 · Питание: PMIC, зарядка, энергопотребление

PMIC (контроллер питания) и зарядка — критичны для юзабельности (без них телефон не живёт автономно и не заряжается корректно). Управление доменами питания, регуляторы, термальные пороги, suspend/resume — всё это PEBEP по downstream-ядру. Качественный power-management на новом SoC под микроядром — отдельная большая задача; на ранних этапах ожидается заметно худшая автономность, чем на стоке.

3.8 · Хранилище (UFS)

UFS-контроллер нужен рано — чтобы грузить образы и писать логи. Драйвер UFS под Zircon (DFv2): инициализация контроллера, UFS HCI, LUN, файловая система (Fuchsia использует свои ФС — minfs/fxfs). Статус PEBEP/ПОРТ — контроллеры UFS относительно стандартизированы (JEDEC), что облегчает задачу относительно полностью кастомных блоков.

3.9 · Связь: Wi-Fi, BT, NFC, GNSS

Wi-Fi/BT (вероятно отдельный комбо-чип) требуют своих драйверов и часто firmware-блобов (используются как есть). Полноценный Wi-Fi-стек под Fuchsia существует, но привязка к конкретному чипу — РЕВЕРС/ПОРТ. NFC и GNSS — вне MVP. Спутниковая связь Pixel 9 (через Exynos 5400) — заведомо вне области.

Радио	Подход	Приоритет
Wi-Fi	ПОРТ драйвера + firmware как есть	высокий (без сети неудобно)
Bluetooth	ПОРТ	средний
NFC	вне MVP	низкий
GNSS	вне MVP	низкий
Satellite SOS	вне области	—

3.10 · Titan M2 и доверенная загрузка

Titan M2 — secure element, обеспечивающий verified boot и аппаратный keystore. Fuchsia поддерживает доверенную загрузку; на финале возможен relock загрузчика со своим ключом (как делают GrapheneOS на Android). Цена relock на этом железе — потеря проприетарных AI-функций Pixel и Play Integrity, что для нас не-цель, поэтому приемлемо. На этапе разработки verified boot обычно ослаблен.

Ядро и субстрат ОС (Fuchsia / Zircon)

Что мы берём как есть из Fuchsia, как форкаем дерево и как организуем разработку в эмуляторе.

4.1 · Архитектура Zircon

Zircon — объектно-капабилити микроядро (наследник Little Kernel). В отличие от классического минимального микроядра, у Zircon порядка 170 системных вызовов — он не стремится к минимализму ради минимализма, вынося в userspace драйверы и сервисы, но оставляя в ядре управление процессами, потоками, виртуальной памятью, IPC и ожиданием состояний объектов. Это даёт малую поверхность атаки ядра и сильную изоляцию.

Все ресурсы представлены объектами ядра, доступ к которым — через хэндлы с правами (rights). Нет «ambient authority»: компонент может делать только то, на что у него есть явный хэндл. Это фундамент модели безопасности всей ОС.

4.2 · Капабилити-модель и хэндлы

Хэндл — это ссылка на объект ядра плюс набор прав. Передача возможностей между компонентами — это передача хэндлов по каналам. Для нашей продуктовой модели это удобно: агенты и интеграции получают строго ограниченные капабилити (например, только чтение конкретного источника), что естественно ложится на приватность и принцип наименьших привилегий.

4.3 · Driver Framework (DFv2)

DFv2 — модель драйверов Fuchsia: драйверы — это компоненты, связываемые с устройствами по bind-правилам, общающиеся через FIDL. Все наши аппаратные драйверы (SoC, дисплей, GPU, модем-транспорт, сенсоры) пишутся против DFv2. Это и есть причина, по которой Linux-драйверы не переносятся: они написаны под совсем другую модель (Linux kernel modules), а DFv2 — нативная цель.

DFv2 предоставляет стабильный драйверный ABI («написал драйвер один раз»), композицию драйверов, и изоляцию (драйвер в отдельном процессе/компоненте), что повышает устойчивость: падение драйвера не роняет ядро.

4.4 · Компонентный фреймворк и FIDL

Fuchsia собрана из компонентов — изолированных единиц с явными возможностями (capabilities), описанными в манифестах, и интерфейсами на FIDL. Нет глобальной файловой системы; у каждого компонента — свой namespace. Это идеально ложится на нашу развязку: платформенные сервисы (хранилище сущностей, телефония, камера) — это компоненты с FIDL-контрактами, которые оболочка и агенты потребляют, не зная деталей реализации.

FIDL — язык описания интерфейсов; по нему генерируются привязки для Rust/C++/др. Контракты между слоями (substrate↔platform↔shell) фиксируются именно как FIDL, что и обеспечивает заменяемость substrate (эмулятор vs железо).

4.5 · Magma — графический субстрат

Magma — графическая подсистема Fuchsia, проводящая границу между системным графическим стеком и драйвером конкретного GPU (аналогична границе DRM/Mesa в Linux, но своя). Наш Mali-драйвер реализуется как Magma-драйвер. Это значит, что открытые наработки Panfrost (Linux DRM/Mesa) дают нам понимание железа (форматы команд, ISA), но не готовый код — код пишется под Magma (ПОРТ).

4.6 · Стратегия форка дерева Fuchsia

Мы форкаем всё дерево Fuchsia (Zircon + userspace + сборка) и ведём свою ветку. В форке: Zircon, DFv2, FIDL, Magma, Scenic/Flatland, Starnix используются как есть; добавляются наши board/драйверы (substrate) и наш продуктовый слой (platform+shell). Прецедент — dahliaOS (форк Fuchsia, умеющий работать и на Zircon, и на mainline Linux), показывающий, что форк осуществим, хотя и экспериментален.

Важно про форк. Форк — лёгкая часть и не blocker: дерево открыто, форкается сегодня. Но форк без bring-up — это просто переименованное дерево Google. Ценность форка — в нашем substrate (Трек В) и продукте (Трек А), а не в самом акте форка.

4.7 · Система сборки (GN/ninja, fx)

Fuchsia собирается через GN/ninja и набор инструментов fx. Сборка тяжёлая (нужны приличные диск/RAM/CPU), официально поддерживается на x86-64 Linux/Mac. Для нашего проекта это означает выделенную сборочную машину/CI; продуктовый слой итеративно собирается и гоняется в эмуляторе на той же машине.

4.8 · Эмулятор FEMU и рабочий цикл разработки

FEMU — эмулятор Fuchsia (на базе QEMU/AEMU) для разработки на x86. Ключ всей стратегии: продуктовый слой (Трек А) разрабатывается, отлаживается и демонстрируется в FEMU без какого-либо железа. Рабочий цикл: правка → `fx build` → запуск в FEMU → отладка Zircon через `ffx/GDB`. Есть публичные разборы (например, статья Александра Попова «A Kernel Hacker Meets Fuchsia») с практикой сборки и отладки Zircon в QEMU.

Таким образом `substrate` в Треке А эмулируется: камера/модем/датчики представлены mock-сервисами с теми же FIDL-контрактами, что и реальные драйверы. Когда Трек В поднимет реальный `substrate`, продукт переезжает без изменений выше FIDL-границы.

GPU и графика

Mali-G715 — самый «решённый» из закрытых блоков благодаря Panfrost, но код всё равно пишется под Magma.

5.1 · Архитектура Mali-G715 (Valhall, CSF)

Mali-G715 — GPU архитектуры Valhall с командным фронтендом CSF (Command Stream Frontend): работа подаётся через очереди команд, которыми управляет прошивка GPU (firmware на встроенном микроконтроллере). Это отличает Valhall-CSF от более старых Mali (Midgard/Bifrost с job-manager) и определяет, какой именно открытый драйвер применим.

Для нас важно: ядро (kernel-side) для CSF-GPU — это Panthor (новый kernel-драйвер), userspace — Panfrost/PanVK в Mesa. То есть Mali-G715 относится к самой современной ветке открытой поддержки Mali, и это удача относительно, например, PowerVR в Pixel 10.

5.2 · Экосистема открытых драйверов Mali

Компонент	Что это	Польза для нас
Panfrost	userspace-драйвер (Mesa) для Mali	источник RE-знаний: командные потоки, ISA
PanVK	Vulkan-драйвер для Mali в Mesa	референс Vulkan-пути для Valhall
Panthor	kernel-драйвер для Valhall-CSF (DRM)	модель взаимодействия с CSF-прошивкой
Tyr	новый kernel-драйвер на Rust (Collabora+Arm+Google)	современный референс; Google причастен

Эти проекты — Linux DRM/Mesa. Они не переносятся в Fuchsia как код, но дают почти всё необходимое понимание железа: форматы командных потоков CSF, ISA шейдеров Valhall, протокол общения с прошивкой. Это и есть содержание статуса ПОРТ для GPU-драйвера: переиспользуем знания, пишем под Magma.

5.3 · Магма и портирование драйвера

Реализация GPU-драйвера на Магма включает: интеграцию с CSF-прошивкой GPU (загрузка firmware-блоба как есть — КАК ЕСТЬ), реализацию подачи командных потоков, управление памятью GPU (GPU MMU), синхронизацию, и компилятор/трансляцию шейдеров. Часть компилятора шейдеров концептуально переиспользуется из понимания Panfrost-ISA, но привязка к Магма-абстракциям — наша.

Реалистичный путь по фазам: (1) headless-инициализация GPU и подача простейших команд; (2) базовый рендеринг/композиция для UI; (3) Vulkan-путь; (4) оптимизация. До стадии 2 UI может работать на программном рендеринге (медленно, но позволяет вести Трек А-на-железе).

5.4 · Методология реверс-инжиниринга GPU

Метод — не дизассемблирование драйвера, а наблюдение интерфейса железа: перехват командных буферов, которые проприетарный драйвер шлёт в GPU (через шим/трейс MMU и регистровых записей), и сопоставление с результатом. Именно так строились Panfrost/Freedreno. Для нас значительная часть этой работы уже сделана сообществом для Linux — мы переиспользуем документированные форматы, а доисследуем лишь специфику нашей ревизии.

Clean-room для GPU. Командные форматы CSF берём из открытой документации Panfrost/Panthor (происхождение задокументировано). Если потребуется доисследование — наблюдаем поведение железа, не дизассемблируем закрытый Mali-блб.

5.5 · Прошивка GPU и блобы

Valhall-CSF требует загрузки подписанной прошивки GPU на встроенный микроконтроллер. Эту прошивку мы используем как есть (КАК ЕСТЬ) — не реверсим и не модифицируем. Аналогично open-драйверы Mali/PowerVR в Linux грузят firmware-блб. Поэтому «ноль блобов» здесь недостижим — реверсится драйвер, а не прошивка.

5.6 · Композитор и вывод на экран

Поверх Магма-драйвера работает Scenic/Flatland — система композиции Fuchsia. Наша оболочка (L6) рисует через Flatland; display-coordinator выводит композированный кадр на дисплей-контроллер (драйвер из L1). Таким образом графический путь замыкается: shell → Flatland → Magma(Mali) → display driver → панель.

Вход (как есть)	Magma, Scenic/Flatland, display-coordinator API, GPU firmware
Порт	Mali-драйвер на Магма (знания Panfrost/Panthor/PanVK/Tyr)
Реверс	специфика ревизии GPU (при необходимости), дисплей-контроллер

Телефония и модем (Exynos 5400)

Модем — чёрный ящик, которым командуют. «Слать команды» посильно; под этим — три недостающих слоя.

6.1 · Обзор Exynos Modem 5400

Exynos Modem 5400 — внешний 5G-модем Samsung поколения 2024 года (поддерживает 3GPP Rel.17, спутниковую NTN-связь). Это самостоятельный процессор со своей RTOS и подписанной baseband-прошивкой. Мы его не реверсим как устройство — мы поднимаем канал к нему и говорим на его управляющем протоколе. Прошивку грузим как есть (КАК ЕСТЬ), радиоповедение не меняем (регуляторика).

6.2 · Стек телефонии целиком

Телефония — слоёный стек от железа к приложению. Снизу: модем и его firmware-блób (как есть). Над ним: boot-секвенс (реверс), link-транспорт (порт), набор команд (реверс), кодек IPC-протокола (порт), RIL/ModemManager-логика (обёртка через Starnix), приложение (с нуля).

Телефонный стек: AP ↔ модем

Модем — чёрный ящик, которым командуют; снизу вверх — от железа к приложениям

Приложение телефонии / SMS / данные (наш UI)

NEW

RIL / ModemManager / ofono — регистрация, звонки, SMS, bearer

WRAP

libsamsung-ipc — кодек IPC-протокола Samsung

PORT

Набор команд Exynos 5400 (RE через BaseMirror из libsec-ril.so)

REV

Link-транспорт SIPC — char-устройство /dev/umts_ipc0 (драйвер под Fuchsia)

PORT

Boot-секвенс: power/GPIO + заливка подписанного бλόба

REV

Модем Exynos 5400 + baseband-прошивка (blob, не трогаем)

ASIS

Параллельная ветка: голос — отдельный аудио-тракт (модем/аудио-DSP), не покрывается командным каналом данных.

Телефонный стек AP↔модем: статус каждого слоя

6.3 · Link-транспорт (SIPC)

Канал AP↔модем у Samsung — это shared-memory протокол (SIPC), экспонируемый через char-устройство (исторически `/dev/umts_ipc0`) поверх драйвера `modem_if`. На Fuchsia такого драйвера нет — его нужно написать как нативный DFv2-драйвер link-транспорта. Это статус ПОРТ: модель и протокол известны из Replicant/Linux, реализация — под Zircon. Без рабочего транспорта команды слать физически некуда — это первый недостающий слой под «отправкой команды».

6.4 · Boot-секвенс модема

Модем не стартует сам: AP подаёт питание, выполняет GPIO-хендшейк, заливает подписанный firmware-блوك по нужным адресам и выводит модем из сброса. Последовательность специфична для каждой модели и недокументирована; восстанавливается методом наблюдения (`strace/dmesg`/исходники ядра) — ровно так это делали для Galaxy S7. Для Exynos 5400 этот секвенс никем публично не описан — статус РЕБЕРС. Каждый пакет в канале форматирован заголовком (`sipc_fmt_hdr`) с собственной длиной, что упрощает парсинг кадров.

6.5 · IPC-протокол (libsamsung-ipc)

`libsamsung-ipc` — свободная реализация модемного IPC-протокола Samsung (проект Replicant). Она содержит device-specific код транспорта между ядром и верхним слоем (RIL) и определения структур протокола. Это наша основа «слать команды» (статус ПОРТ): код Linux-форменный, поэтому либо переносим, либо гоняем под Starnix. Ограничение: библиотека покрывает модемы ~2011 года (i9300, Nexus S и т.п.) и не поддерживает даже Galaxy S7 — современный Exynos 5400 потребует расширения.

6.6 · RE набора команд (BaseMirror)

Конкретный набор команд Exynos 5400 в открытых либах отсутствует. Здесь помогает свежий инструментарий BaseMirror (2024–2026): он автоматически реверсит baseband-команды из вендорских RIL-библиотек Samsung Exynos (`libsec-ril.so`), применён к 28 моделям Exynos. Это означает, что мы стартуем не с нуля, а извлекаем командный набор из RIL-бинаря самого Pixel 9 — статус РЕБЕРС, но с инструментальной поддержкой.

Почему это второй и третий недостающие слои. Под «отправкой команды» лежат: (1) транспорт (порт), (2) boot-секвенс (реверс), (3) набор команд именно этого модема (реверс через BaseMirror). «Слать команды» — посильный верхний слой; вся сложность — в этих трёх слоях под ним.

6.7 · RIL / ModemManager / ofono

Над IPC-кодеком — слой телефонии (регистрация в сети, исходящие/входящие звонки, SMS, поднятие data-bearer). В мире Linux это ModemManager/ofono + libsamsung-ril. Мы запускаем эту логику через Starnix (статус ОБЕРНУТЬ), а наружу к продукту экспонируем телефонию как FIDL-сервис платформенного слоя. Это избавляет от переписывания зрелой телефонной логики под Zircon.

6.8 · Голосовой тракт

Голос — отдельная и более тяжёлая задача, чем данные/SMS. Аудио звонка идёт не через канал данных, а через отдельный путь (аудио-DSP/маршрутизация PCM между модемом и аудиоподсистемой). Это требует и аудио-драйвера, и корректной маршрутизации, и синхронизации с состоянием звонка. Голос вынесен в отдельную веху, которой допустимо провалиться: данные и SMS могут работать без рабочего голоса.

6.9 · SMS, данные, регистрация, экстренные вызовы

Функция	Зависит от	Реалистичность
Регистрация в сети	транспорт+boot+команды	достижимо (база)
SMS	протокол+кодировки (вкл. кириллицу)	достижимо
Мобильные данные	bearer/APN через RIL	достижимо
Голосовые звонки	аудио-тракт	веха с риском провала
Экстренные вызовы	сертифицированное поведение	вне гарантий; не полагаться
VoLTE/5G NR	IMS/современный стек	сложно; поздняя веха

Отдельный регуляторный и безопасностный нюанс: на экстренные вызовы и сертифицированное радиоповедение полагаться нельзя — самодельный телефонный стек не сертифицирован.

6.10 · Регуляторные аспекты

Ключевое разграничение: отправка штатных управляющих команд модему — легитимна и не является проблемой. Регулируется и запрещено к модификации радиоповедение/прошивка baseband (она и держит сертификацию). Поэтому прошивку грузим как есть, не трогаем, и не пытаемся менять РЧ-параметры — командуем только через штатный интерфейс.

Камера и вычислительная фотография

Самый открытый кусок: алгоритмы Pixel опубликованы. Обработка переносима; потолок — ручной тюнинг сенсора.

7.1 · Камерное железо

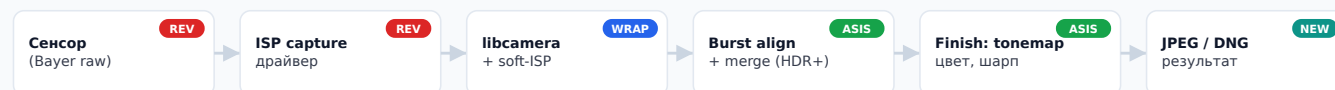
Камерная подсистема — сенсоры (Bayer raw), ISP (image signal processor) и связанные шины. ISP и его тюнинг — самые охраняемые части; вычислительная фотография Google живёт в закрытом приложении (GCam), а не только в драйвере. Поэтому реверс драйвера даёт захват кадров, но не «магию» Pixel.

7.2 · Конвейер захвата и обработки

Конвейер: сенсор отдаёт Bayer-raw → ISP capture-драйвер доставляет кадры → libcamera (+ software-ISP при отсутствии доступа к hw-ISP) делает демозаик/базовую обработку → HDR+ пайплайн выравнивает и сливает бурст → финиш (тонмаппинг, цвет, шарп) → JPEG/DNG.

Конвейер камеры: от сенсора до снимка

Обработка (HDR+) — переносима и берётся как есть; потолок качества — ручной тюнинг сенсора



Тюнинг под сенсор (С НУЛЯ): CCM · black level · профиль шума · AWB/AE

Влияет на все стадии. Это, а не алгоритм, отделяет «достойно» от Pixel-grade. Ручная работа на каждый модуль.

Чего не будет без закрытого GCam: Night Sight, Super Res Zoom, Real Tone, Magic Eraser — это learned-модели и годы тюнинга поверх HDR+.

Конвейер камеры: статусы стадий и роль тюнинга

7.3 · libcamera и software-ISP

libcamera — открытый камерный фреймворк (преемник прямого V4L2) с программным ISP для устройств, где аппаратный ISP недоступен. На Qualcomm-устройствах сообщество как раз использует software-ISP libcamera, и цветокоррекция (CCM) «на подходе» — это убирает зелёный оттенок и приближает результат к Android. Мы запускаем userspace libcamera через Starnix (ОБЕРНУТЬ), а capture-драйвер пишем нативно (РЕВЕРС).

7.4 · HDR+ пайплайн

HDR+ — «движок» камеры Pixel, и он переоткрыт. Базовая статья (Hasinoff et al., 2016) публична; есть рабочие открытые реализации: `hdr-plus` (C++/Halide, T. Brooks), `hdrplus-python` (рецензированная, IPOL 2021), `hdr-plus-pytorch` (GPU). Ядро (выровнять бурст raw → слить/убрать шум → тонмаппинг) полностью воспроизводимо. Этот слой берём КАК ЕСТЬ: он OS-агностичен — просто код «raw-кадры → картинка», и запускается хоть под Starnix, хоть на десктопе для отладки.

Берём как есть	hdr-plus / Halide / датасет HDR+
Обёртываем	libcamera userspace под Starnix
Реверсим	сенсор + ISP capture-драйвер
С нуля	тюнинг под сенсор; финальное приложение камеры

7.5 · Тюнинг под сенсор

Настоящий потолок качества — не алгоритм, а ручной тюнинг под конкретный сенсор: цветовые матрицы (CCM), black level, профиль шума, авто-баланс белого и экспозиция. Именно это делает HDR+ красивым на конкретном модуле, и именно это сообщество (например, `postmarketOS` для Pixel 3A) доделывает руками месяцами. Статус С НУЛЯ — на каждый сенсор своя калибровка.

7.6 · Нейросетевые/learned подходы

Поверх HDR+ существует открытая исследовательская линия по low-light и learned-ISP («Learning to See in the Dark» и последователи) — обучаемые модели raw→RGB. Их можно подключать как дополнительные стадии, но это исследовательский трек, а не база, и он не заменяет тюнинг.

7.7 · Анализ потолка качества

Итоговая честная оценка: камера ПОЕДЕТ, но на уровне postmarketOS («достойно, по-ретро»), а не Pixel-on-Android. Чего не будет без закрытого GCam и фирменного тюнинга: Night Sight, Super Res Zoom, Real Tone, Magic Eraser — это learned-модели и годы тюнинга поверх HDR+. Открытые наработки дают фундамент, не полировку.

Возможность	Статус в нашей ОС
Базовое фото (raw→JPEG)	достижимо
HDR+ бурст (динамический диапазон, шум)	достижимо (открытый код)
Видео	ограниченно (зависит от ISP/энкодера)
Night Sight / Super Res Zoom	недостижимо (закрытый GCam)
Цвет «как Pixel»	зависит от ручного тюнинга; не гарантирован

Совместимость с Linux/Android (Starnix)

Единственный модуль, который радикально упрощает всё выше: превращает часть «портов» в «обёртки».

8.1 · Архитектура Starnix

Starnix — рантайм, реализующий интерфейс Linux-ABI (UAPI) поверх Zircon в userspace, без виртуальной машины: он транслирует системные вызовы Linux-программы в подсистемы Fuchsia. Протестирован с glibc- и bionic-бинарями, имеет интерактивный Android-shell, и реализует binder (IPC Android). Это и есть путь Fuchsia для запуска Linux/Android-userspace.

Роль Starnix: что нативно, а что оборачиваем

Starnix даёт Linux-ABI и binder поверх Zircon — превращает часть «портов» в «обёртки»

Нативно (DFv2 / Magma / FIDL)

Драйверы железа: SoC, дисплей, USB, PMIC

REV

GPU-драйвер на Magma

PORT

Modem link-транспорт (SIPC)

PORT

Сенсор/ISP capture-драйвер

REV

Shell + композитор (Scenic/Flatland)

NEW

Кернел-уровень: Starnix не помогает, нужен нативный драйвер.

Через Starnix (Linux/bionic-бинари)

libsamsung-ipc / libsamsung-ril

PORT

ModemManager / ofono

WRAP

libcamera userspace + soft-ISP

WRAP

hdr-plus / Halide-пайплайн

ASIS

KDE Connect / OpenDrop / uxplay

WRAP

Userspace-уровень: гоняем существующий код без переписывания под Zircon.

Что нативно (DFv2/Magma), а что через Starnix

8.2 · Запуск Linux/bionic-бинарей

Ключевое следствие для проекта: Starnix позволяет гонять немодифицированный Linux/bionic userspace. Значит, Linux-форменные компоненты (libsamsung-ipc/libsamsung-ril, ModemManager/ofono, libcamera userspace, hdr-plus, KDE Connect/OpenDrop/uxplay) можно ОБЕРНУТЬ — запустить под Starnix, а не переписывать под Zircon. Это превращает значительную часть потенциальных «портов» в «обёртки» и резко снижает объём.

8.3 · binder в Starnix

Поскольку Android-userspace опирается на binder, Starnix реализует binder (включая remote-binder и оптимизации производительности). Это важно, если мы захотим гонять Android-userspace-компоненты телефонии/камеры, которые общаются через binder, не имея Linux-ядра с binder-драйвером под собой.

8.4 · Где Starnix не помогает

Граница чёткая: Starnix — это userspace. Всё, что является kernel-уровневым драйвером (bring-up SoC, GPU-драйвер на Magma, link-транспорт модема, сенсор/ISP capture, дисплей), Starnix не закрывает — это нативные DFv2/Magma-драйверы (РЕВЕРС/ПОРТ). Иными словами, Starnix снимает нагрузку с userspace-слоёв, но не с драйверного фундамента.

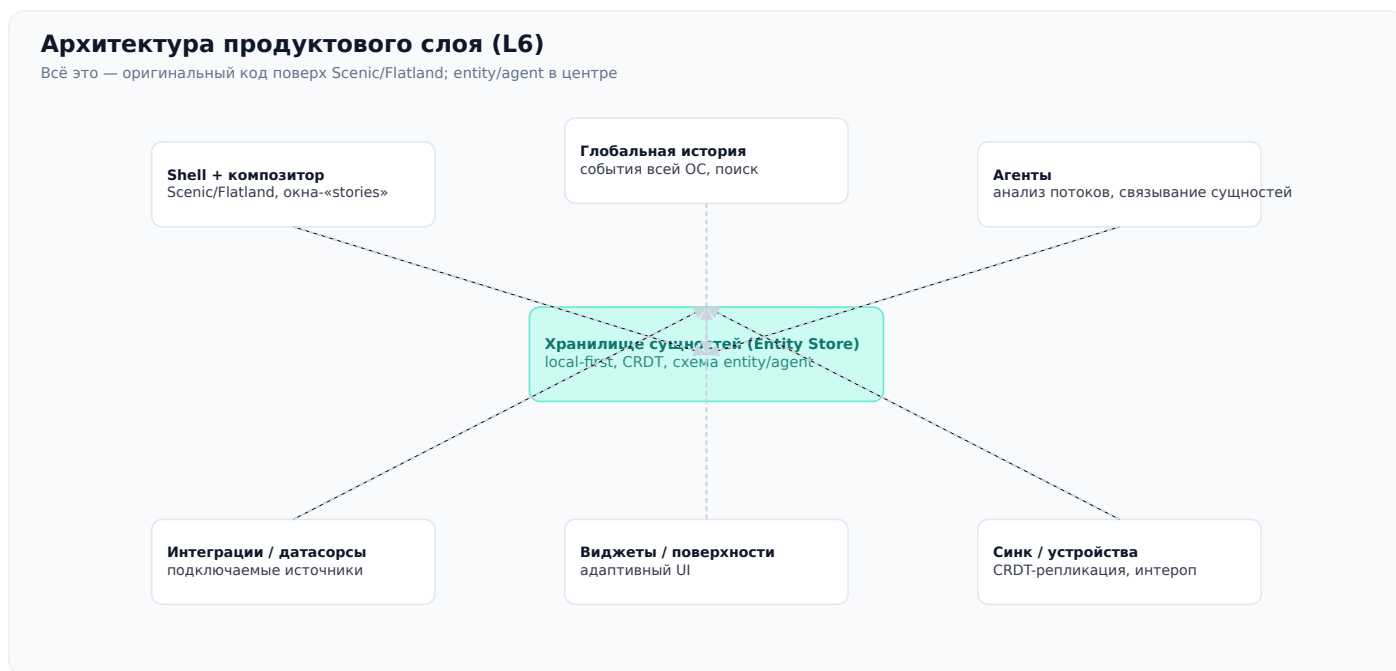
Множитель сложности. Starnix — главный множитель эффективности всего проекта выше драйверного уровня. Без него телефония/камера/интероп потребовали бы переписывания под Zircon; с ним — это в основном обёртки.

Userland / продуктовый слой

Само видение ОС — и самый ОС-независимый слой: строится в FEMU уже сейчас. Почти всё здесь — оригинальный код.

9.1 · Архитектура продукта

Продуктовый слой — это хранилище сущностей (Entity Store) в центре, вокруг которого работают агенты, оболочка/композитор, глобальная история, интеграции/виджеты и синхронизация. Всё это — оригинальный код (С НУЛЯ) поверх Scenic/Flatland и платформенных FIDL-сервисов.



Продуктовый слой: хранилище сущностей в центре, модули вокруг

9.2 · Shell и композитор (Scenic/Flatland)

Оболочка рисует через Flatland (систему композиции Fuchsia, берётся как есть как framework). Концепция поверхностей — не сетка иконок, а «истории»/поверхности, собираемые вокруг текущей сущности или задачи. Оболочка управляет окнами-«stories», глобальным поиском и точкой входа в историю.

9.3 · Модель entity/agent

Ядро продукта — типизированное хранилище сущностей со схемой (проект/задача/документ/человек/место/событие/сообщение/устройство) и рёбрами между ними. Агенты — фоновые компоненты с ограниченными капабилити, которые: извлекают сущности из источников, дедуплицируют и связывают их, поддерживают историю, и предлагают действия. Каждый агент получает строго очерченный доступ (только нужный источник) — это естественно ложится на капабилити-модель Zircon.

Сущность	Примеры рёбер	Источники
Человек	участвует в, автор, контакт	почта, контакты, мессенджеры
Проект/Задача	содержит, блокирует, относится к	трекеры, заметки, календарь
Документ	порождён из, упомянут в	файлы, облака, заметки
Событие/Место	происходит в, рядом с	календарь, геоданные, датчики
Сообщение	в тредде, от/к	почта, мессенджеры

9.4 · Глобальная история

Глобальная история — журнал событий всей ОС (как история браузера, но для системы целиком): что открывалось, создавалось, с кем и где. Это и навигационный инструмент (вернуться к контексту), и субстрат для агентов (связывание по времени/контексту). Требует продуманной модели приватности (см. 10.3) и быстрого локального поиска.

9.5 · Local-first данные и CRDT-синк

Данные — local-first: основное состояние живёт на устройстве, синхронизация опциональна и идёт через CRDT (бесконфликтные реплицируемые типы), что даёт работу офлайн и автоматическое слияние между устройствами. CRDT-библиотеки берём как есть; схема и интеграция — наши. Это развязывает устройство и облако: облако — лишь транспорт/резерв, а не источник истины.

9.6 · Интеграции, датасорсы, виджеты

Интеграции — подключаемые источники данных (почта, календарь, мессенджеры, облака, локальные сенсоры), реализуемые как агенты с капабилити. Виджеты/поверхности — адаптивные UI-элементы, отображающие сущности и действия. Расширяемость — через стабильный контракт агента/виджета.

9.7 · Apple-интероп (AirDrop/AirPlay/буфер)

Интероперабельность с Apple/Linux — через открытые реализации, запускаемые под Starnix (ОБЕРНУТЬ): OpenDrop (AirDrop поверх AWDL), uxplay/shairport-sync (AirPlay), KDE Connect/GSConnect (кросс-девайс буфер и синк, есть и клиенты под Mac). На Linux-userland это ставится как обычные компоненты — иронично, но проще, чем пробивать песочницу Android.

Функция	Открытая реализация	Статус
AirDrop	OpenDrop (AWDL)	ОБЕРНУТЬ
AirPlay	uxplay / shairport-sync	ОБЕРНУТЬ
Общий буфер/синк	KDE Connect / GSConnect	ОБЕРНУТЬ

9.8 · UX-парадигма

UX строится вокруг текущей сущности/задачи, а не вокруг приложений: пользователь работает с проектом/документом/человеком, а нужные действия и источники подтягиваются агентами. Глобальный поиск и история — первичные навигационные примитивы. Это и есть отличие от стоковых мобильных ОС и одновременно главный риск принятия — парадигма непривычна.

Сквозные аспекты

Безопасность, обновления, приватность, производительность, отладка, тестирование — поверх всех слоёв.

10.1 · Модель безопасности

Основа — капабилити-модель Zircon (нет ambient authority; доступ только по явным хэндлам) и компонентная изоляция (sandbox, отсутствие глобальной ФС, явные namespaces). Драйверы изолированы (падение драйвера не роняет ядро). Аппаратно — MTE (ловит классы багов памяти) и Titan M2/verified boot с возможностью relock. Агенты и интеграции получают минимальные капабилити под свой источник.

10.2 · Обновления и OTA

Fuchsia спроектирована вокруг обновляемости: компоненты идентифицируются URL и резолвятся/обновляются по требованию (пакеты «как веб-страницы»). Для нашей ОС это означает A/B-обновления substrate и независимое обновление продуктовых компонентов. На этапе разработки OTA не приоритет; в зрелой версии — обязательный механизм безопасных откатываемых обновлений.

10.3 · Приватность

Приватность встроена архитектурно: local-first (данные на устройстве), капабилити (агент видит только свой источник), отсутствие телеметрии, опциональный синк под контролем пользователя. Глобальная история — мощный, но чувствительный субстрат: требует шифрования на устройстве, чётких границ доступа агентов и возможности исключать/забывать данные.

10.4 · Производительность и энергопотребление

Честное ожидание: на ранних этапах bring-up автономность и плавность будут хуже стока. Power-management на новом SoC под микроядром (suspend/resume, тактовые/тепловые политики) — отдельная большая задача (L1). GPU-ускорение появляется поэтапно (до этого — программный рендеринг). Микроядро добавляет IPC-оверхед, который компенсируется архитектурой, но требует внимания к горячим путям.

10.5 · Наблюдаемость и отладка

Инструменты Fuchsia: ffx (управление/диагностика), логирование (archivist), трассировка, отладка Zircon через GDB/QEMU. На железе критичен ранний UART-консоль. Для продукта — трассировка агентов и истории. Отладка bring-up без документации опирается на наблюдение интерфейсов (как в RE).

10.6 · Стратегия тестирования

Уровень	Что тестируем	Где
Юнит/компонент	логика агентов, хранилище, синк	FEMU/CI
Интеграция (FIDL)	контракты <code>substrate↔platform↔shell</code>	FEMU с mock-substrate
Драйверы	bring-up блоков	на железе + стенды
Совместимость (Starnix)	Linux/bionic-компоненты	FEMU/железо
Сквозное	сценарии продукта	FEMU; затем железо

Принцип: всё, что можно протестировать в FEMU с mock-substrate, тестируется там (быстро, без железа); железо-зависимое — отдельным контуром.

10.7 · Сборка, CI и воспроизводимость

Сборка через GN/ninja/fx на выделенной машине; CI прогоняет юнит/интеграционные тесты в FEMU. Форк пиннит ревизию upstream Fuchsia (защита от движущейся мишени) с регулярными управляемыми апдейтами. Артефакты substrate (board, драйверы) и продукта собираются отдельно, отражая развязку.

Дорожная карта и исполнение

Фазы, вехи, оценки трудозатрат, критический путь и минимальные жизнеспособные конфигурации.

11.1 · Рoadmap (фазы)

Рoadmap задаётся фазами, не календарём: на недокументированном железе разброс сроков велик. Трек А движется быстро и предсказуемо в FEMU; Трек В — медленно и с высоким разбросом, сильно завися от удачи RE.



Фазовый roadmap двух треков (длительности условны)

11.2 · Трек А: вехи (FEMU)

- A1 — Форк Fuchsia, сборка, запуск в FEMU, каркас компонентов.
- A2 — Оболочка и композитор на Scenic/Flatland; базовые поверхности.
- A3 — Хранилище сущностей и первые агенты; модель entity/agent.
- A4 — Глобальная история + local-first данные + CRDT-синк.
- A5 — Интеграции/датасорсы/виджеты + Apple-интероп через Starnix.

11.3 · Трек В: вехи (железо)

- B1 — Zircon бутится на Tensor (UART, ядро SoC, память).
- B2 — Дисплей + ввод (тач/сенсоры): первый видимый UI на железе.
- B3 — GPU-драйвер на Magma (ускоренная композиция).
- B4 — Модем: транспорт → boot → команды → данные/SMS (голос — отдельно).
- B5 — Камера: capture-драйвер → libcamera → HDR+ → тюнинг.
- B6 — Интеграция продукта (A) поверх реального substrate (B).

11.4 · Оценки трудозатрат по модулям

Модуль	Статус	Трудозатраты	Риск/неопределённость
Форк + сборка + FEMU	ФОРК	S	низкий
Shell/композитор	С НУЛЯ	M	низкий
Entity/Agent + хранилище	С НУЛЯ	L	средний (дизайн)
История + синк (CRDT)	С НУЛЯ	M	средний
Интеграции/интероп	С НУЛЯ/ОБЕРНУТЬ	M	низкий
Zircon на Tensor	РЕВЕРС	XL	очень высокий
Дисплей/ввод	РЕВЕРС/ПОРТ	L	высокий
GPU на Magma	ПОРТ	XL	высокий
Модем: транспорт+boot+команды	ПОРТ/РЕВЕРС	XL	очень высокий
Голос модема	РЕВЕРС	L	очень высокий (может не закрыться)
Камера: capture+тюнинг	РЕВЕРС/С НУЛЯ	XL	высокий
Starnix-обёртки	ОБЕРНУТЬ	M	средний (ABI)

Шкала условная: S — дни/недели, M — недели/месяцы, L — месяцы, XL — месяцы/годы при удачном RE.

11.5 · Критический путь и зависимости

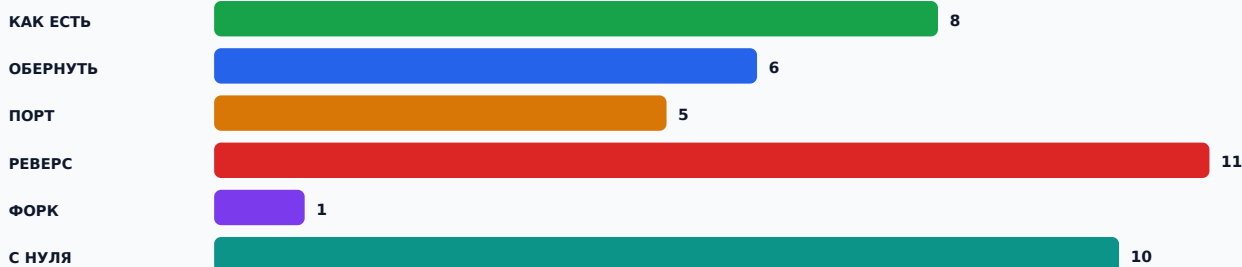
Критический путь Трека В: B1 (Zircon на Tensor) блокирует всё остальное на железе — без загрузки и базовой периферии нет ни дисплея, ни GPU, ни модема. Дисплей (B2) — ранний приоритет ради видимого прогресса. Модем (B4) и камера (B5) — самые рискованные и могут идти параллельно после B1-B3. Трек А зависимостей от железа не имеет вовсе — в этом вся идея развязки.

11.6 · Распределение по статусам

Сводное распределение модулей по статусам показывает профиль проекта: доминируют РЕВЕРС и С НУЛЯ — это и есть мера амбициозности. КАК ЕСТЬ/ОБЕРНУТЬ концентрируются в верхних слоях (благодаря Starnix), РЕВЕРС — в железе.

Распределение модулей по статусам

Ориентировочная оценка по карте (пере)использования



Реверс и «с нуля» доминируют — это и есть мера амбициозности проекта.

Распределение модулей по статусам (оценка)

11.7 · Минимальные жизнеспособные конфигурации

Чтобы проект давал ценность раньше, определены урезанные конфигурации:

Конфигурация	Что входит	Что отброшено
MVP-A (продукт в FEMU)	весь L6 + Starnix-интероп в эмуляторе	всё железо
MVP-B0 (железо живо)	Zircon+дисплей+ввод+Wi-Fi на Pixel 9	GPU-ускор., модем, камера
MVP-B1 (с данными)	+ модем: данные/SMS (без голоса)	голос, камера
MVP-B2 (с камерой)	+ базовое фото (HDR+, без тюнинга-полировки)	голос, флагман-качество
Полная	всё, включая голос и тюнинг	—

Прагматичный порядок: довести MVP-A до демонстрируемого продукта, затем MVP-B0 на железе (Wi-Fi-«планшет» без телефонии), и только потом погружаться в модем/камеру.

Сравнение базовых платформ

Почему именно Fuchsia, чем она отличается от альтернатив и при каких условиях выбор стоит пересмотреть.

12.1 · postmarketOS (mainline Linux)

postmarketOS — настоящий Linux на mainline-ядре, без Android под собой, с целью 10-летнего жизненного цикла устройств. Сильные стороны: чистая апстрим-база, любой UI (Phosh/Plasma Mobile/GNOME/Sxmo), огромная инфраструктура bring-up (pmbootstrap, deviceinfo) и сообщество. Слабые: работает хорошо лишь там, где SoC поддержан mainline; камера и модем — самые частые точки разочарования (на многих устройствах камера only-still или не работает, модем надёжен лишь на некоторых).

Для нашего видения postmarketOS — самый прагматичный путь, если отказаться от микроядра: продуктовый слой можно построить поверх обычного Linux-ядра и получить 95% задуманного без многолетней возни с драйверами под микроядро. Но это не даёт капабилити-микроядра и компонентной модели, ради которых выбрана Fuchsia.

12.2 · Haliu / Droidian

Haliu — слой, позволяющий GNU/Linux-системе использовать проприетарные Android-драйверы на устройстве с предустановленным Android, через libhybris и контейнер с Android-HAL поверх слегка модифицированного downstream-ядра. На нём построены Ubuntu Touch, SailfishOS и Droidian (Debian на телефоне).

Сильная сторона для камеры/модема: они идут через настоящий Android-HAL, поэтому работают лучше, чем на mainline. Слабая: всегда тянет проприетарные блобы и замороженное вендорское ядро, и не даёт ни микроядра, ни своего kernel. Для «рабочая камера/звонки и быстро строить продукт» Droidian — самый прагматичный вариант; но он несовместим с целью «своя микроядерная ОС».

12.3 · Genode / Sculpt (микроядро на телефоне сегодня)

Genode — компонентная капабилити-ОС поверх семейства микроядер (включая seL4), а Sculpt — её готовый дистрибутив, который разработчики используют как ежедневную систему на PinePhone, с песочными драйверами и виртуалками; в свежих релизах добавлена опция seL4 на ARM64. Это единственный способ «потрогать микроядерный телефон» уже сегодня.

Для нашего проекта Genode — ценный ориентир и альтернатива: если важнее «микроядро на реальном телефонном железе сейчас», чем продуктовая модель Fuchsia, то PinePhone+Sculpt даёт это без bring-up-ада (там железо открытое). Минус — слабая камера/железо PinePhone и иная экосистема.

12.4 · Сводное сравнение

Критерий	Fuchsia (наш выбор)	postmarketOS	Halium/Droidian	Genode/Sculpt
Ядро	микроядро Zircon	монолит Linux	монолит Linux+Android	микроядро (NOVA/seL4)
Камера/модем	с нуля (тяжело)	слабо	хорошо (Android-HAL)	слабо (PinePhone)
Микроядро/капабилити	да	нет	нет	да
Bring-up на флагмане	очень тяжело	тяжело	умеренно	н/д (PinePhone)
Продуктовая модель	наша, с нуля	любой UI	любой UI	Genode-специфична
Готово сегодня на телефоне	нет	частично	да	да (PinePhone)

12.5 · Почему всё же Fuchsia и когда пересмотреть

Fuchsia выбрана осознанно: микроядро + капабилити + компонентная модель + Starnix + entity/agent-родственный дизайн перевешивают цену bring-up. Пересмотреть стоит, если: (а) приоритет смещается к «работает на телефоне скоро» — тогда Droidian; (б) к «микроядро на железе сейчас» — тогда Genode на PinePhone; (в) к «максимум открытости и долговечности» — тогда postmarketOS. Эти ветки сохранены как явные развилки.

Детализация bring-up драйверов (L1)

Разбор аппаратных блоков Tensor G4, которые нужно поднять под Zircon, в порядке приоритета.

13.1 · UART и ранний консоль

Самый первый и критичный блок: рабочий debug-UART. Без раннего консоль отладка bring-up почти невозможна — нет способа увидеть, где именно ядро зависает. Нужно найти физический UART (часто на тестовых точках/USB-C альтрежиме), завести минимальный драйвер вывода до полной инициализации, и связать его с ранним логированием Zircon. Статус РЕБЕРС (базовые адреса UART из downstream-ядра).

13.2 · GIC и прерывания

GIC (Generic Interrupt Controller) — обязателен для любой периферии, работающей по прерываниям. Инициализация распределителя и CPU-интерфейсов, маршрутизация SPI/PPI, привязка к модели прерываний Zircon. Относительно стандартизирован (ARM GIC), что снижает неопределённость относительно полностью кастомных блоков.

13.3 · Тактирование (clocks) и pinctrl

Тактовые домены (CCF-аналог) и pin-контроллер задают, какие блоки получают такты и как мультиплексируются пины. Это инфраструктура, от которой зависят почти все драйверы: без корректного тактирования контроллер не отвечает, без pinctrl — не подключён к нужным линиям. Восстанавливается из downstream-ядра; реализуется как Zircon-драйверы тактирования/pinctrl. Статус РЕБЕРС, объём — большой из-за количества доменов.

13.4 · PMIC и регуляторы

PMIC управляет регуляторами питания доменов. Нужны: драйвер PMIC (часто по I2C/SPMI), описание регуляторов и их включение в правильном порядке. Ошибки здесь приводят к тому, что блок «мёртв» (не запитан) — частая причина зависаний bring-up. Статус РЕБЕРС.

13.5 · Suspend/resume и энергопотребление

Корректные suspend/resume и тепловые/тактовые политики определяют автономность. На новом SoC под микроядром это отдельная большая задача; на ранних этапах ожидается заметно худшая батарея, чем на стоке (телефон может не входить в глубокий сон). Реализуется поверх clocks/PMIC; статус РЕБЕРС, низкий приоритет до базовой работоспособности.

13.6 · Дисплей: DSI, панель, инициализация

Дисплейный путь: контроллер (DSI), драйвер панели (часто Samsung AMOLED со своей последовательностью команд инициализации — длинный, специфичный блок команд), интеграция с display-coordinator и Flatland. Это первый видимый результат, поэтому ранний приоритет после консоли и базовой инициализации. Статус РЕБЕРС/ПОРТ (последовательность панели — из downstream).

13.7 · UFS подробнее

UFS нужен рано (грузить образы, писать логи). Драйвер: инициализация UFS Host Controller Interface, link startup, конфигурация LUN, очереди команд. Поверх — файловые системы Fuchsia (minfs/fxfs). UFS относительно стандартизирован (JEDEC/UFSHCI), поэтому статус РЕБЕРС/ПОРТ с меньшей неопределённостью, чем у кастомных блоков.

13.8 · Wi-Fi / BT: чип, firmware, стек

Wi-Fi/BT — вероятно отдельный комбо-чип, требующий своего драйвера и firmware-блоба (используется как есть). У Fuchsia есть Wi-Fi-стек (wlan), но привязка к конкретному чипу — ПОРТ/РЕБЕРС. Wi-Fi — высокий приоритет: без сети устройство малополезно даже как «планшет» на этапе MVP-B0. BT — средний приоритет.

Контракты слоёв (FIDL)

Как развязка substrate/platform/shell фиксируется стабильными интерфейсами, и как это даёт замену substrate.

14.1 · Принцип контрактов

Развязка работает только если границы между слоями зафиксированы стабильными контрактами. Во Fuchsia это FIDL-интерфейсы. Продукт зависит не от реализации substrate, а от его FIDL-контракта; поэтому один и тот же продукт собирается и в FEMU (mock-substrate), и на железе (реальные драйверы), без изменений выше границы.

14.2 · Контракт substrate → platform

Платформенные сервисы потребляют железо через FIDL. Примерные интерфейсы (эскиз):

Камера	CaptureStream: подписка на кадры (raw/обработанные), управление режимом
Модем	ModemControl: послать IPC-команду, подписка на события (звонок/SMS/регистрация)
Сенсоры	SensorFeed: поток показаний с метками времени
Дисплей	через display-coordinator/Flatland (стандарт Fuchsia)
GPU	через Magma (стандарт Fuchsia)

Ключевое: эти контракты определяются нами на уровне platform, а их реализация на железе (Трек B) и mock в FEMU (Трек A) — взаимозаменяемы.

14.3 · Контракт platform → shell

Оболочка и агенты потребляют платформенные сервисы данных через FIDL:

EntityStore	CRUD сущностей и рёбер, подписки на изменения
History	запись/запрос событий глобальной истории
ActionBus	предложение/выполнение действий над сущностью
Integration	регистрация агента-источника с капабилити
Sync	состояние синхронизации устройств

14.4 · Mock-substrate для FEMU

Для Трека A substrate эмулируется компонентами, реализующими те же FIDL-контракты: mock-камера отдаёт тестовые кадры/бурсты, mock-модем имитирует события телефонии, mock-сенсоры — синтетические данные. Это позволяет полноценно разрабатывать и тестировать продукт без железа, а затем заменить mock на реальные драйверы.

Модель данных entity/agent

Схема сущностей, жизненный цикл агентов, запросы, дедупликация и синхронизация — ядро продукта.

15.1 · Схема сущностей и рёбер

Хранилище — типизированный граф: узлы-сущности с атрибутами и типизированные рёбра. Базовые типы сущностей и их обязательные/опциональные поля задаются схемой; рёбра несут тип и метаданные (время, источник, уверенность). Схема версионится, чтобы агенты и синк могли эволюционировать без потери данных.

Элемент	Поля (пример)
Person	имя, идентификаторы (email/телефон/handle), аватар
Project/Task	заголовок, статус, дедлайн, ссылки
Document	заголовок, тип, путь/URL, контент-хэш
Event/Place	время, геопозиция, участники
Message	тред, отправитель, получатели, текст/вложения
Edge	тип, from, to, время, источник, confidence

15.2 · Жизненный цикл агента

Агент — компонент с ограниченными капабилити (только свой источник). Жизненный цикл: регистрация (объявляет источник и требуемые капабилити) → подключение (получает хэндлы доступа) → извлечение (тянет данные, нормализует в сущности) → связывание (создаёт рёбра, дедуплицирует) → обновление истории → предложение действий. Агент изолирован: его сбой не роняет систему, а доступ строго очерчен.

15.3 · Запросы и индексация

Над графом нужны быстрые локальные запросы: по типу, по ребру, по времени, полнотекстовый поиск (для истории и сущностей). Реализуется локальным индексом (например, встраиваемое хранилище + полнотекстовый индекс). Запросы — основной примитив и для UI (поверхности вокруг сущности), и для агентов (связывание по контексту).

15.4 · Слияние и дедупликация

Один человек может приходить из почты, контактов и мессенджера как разные записи. Дедупликация связывает их в одну сущность по идентификаторам и эвристикам, сохраняя провенанс (из какого источника пришёл атрибут) и уверенность. Это деликатная задача: ложные слияния разрушают доверие, поэтому слияния обратимы и аудируемы.

15.5 · CRDT и синхронизация подробнее

Состояние реплицируется между устройствами через CRDT: каждое устройство меняет локально, изменения сливаются без конфликтов и без центрального источника истины. Облако (если используется) — лишь транспорт/резерв. Это даёт офлайн-работу и устойчивость. Тонкости: семантика слияния для разных типов (счётчики, множества, тексты), компактизация истории операций, шифрование на устройстве.

Модем: глубокое погружение

Детали SIPC-кадра, ландшафт протоколов, интеграция ModemManager и тестовый путь телефонии.

16.1 · Формат кадра SIPC

Каждый пакет в канале AP↔модем форматирован заголовком фиксированной структуры (`sipc_fmt_hdr`), несущим собственную длину, что упрощает парсинг потока: читаем заголовок, по длине вычитываем тело. Декодирование тел (команды/события) — задача верхнего слоя (`libsamsung-ipc/libsamsung-ril`). Наблюдением за работающим модемом (сопоставляя байты на канале с диагностикой RIL) можно восстанавливать определения, не покрытые открытыми либами.

16.2 · Ландшафт протоколов: QMI / MBIM / AT — и почему Samsung свой

В мире модемов есть стандартизированные управляющие интерфейсы: AT-команды (легаси), QMI (Qualcomm), MBIM (стандарт USB-IF). Устройства на Qualcomm с открытыми интерфейсами (например, PinePhone с Quectel) поэтому относительно «просто работают». Samsung Exynos использует собственный SIPC поверх `shared-memory` — именно поэтому Exynos-модемы болезненны даже на Linux и требуют `libsamsung-ipc`, а не стандартного ModemManager-плагины «из коробки».

16.3 · ModemManager / ofono и плагины

ModemManager/ofono — зрелые телефонные демоны, абстрагирующие модем за плагины. Стратегия: гонять их под Starnix, подключив к нашему транспорту/`libsamsung-ipc` (через соответствующий плагин/RIL-мост). Наружу к продукту экспонируем ModemControl-сервис (FIDL). Это переиспользует зрелую логику регистрации/звонков/SMS вместо переписывания.

16.4 · Состояния звонка и маршрутизация

Звонок — конечный автомат (`idle` → `dialing/ringing` → `active` → `held` → `ended`) с событиями от модема и аудио-маршрутизацией. Сложность голоса в том, что аудио-тракт отдельный от данных: PCM маршрутизируется между модемом и аудиоподсистемой через аудио-DSP, что требует аудио-драйвера и синхронизации с состоянием звонка. Это и делает голос самой рискованной вехой.

16.5 · Тестовый путь телефонии

Прагматичный порядок отладки: (1) поднять транспорт и загрузить модем (проверка по сырым кадрам); (2) регистрация в сети; (3) SMS (включая корректные кодировки, в т.ч. кириллицу); (4) мобильные данные (APN/bearer); (5) голос — отдельной вехой. На каждом шаге — наблюдение канала и сверка с определениями `libsamsung-ril/BaseMirror`.

Камера: глубокое погружение

Стадии ISP-конвейера, цветовая наука/тюнинг, raw/DNG и детали HDR+.

17.1 · Стадии ISP-конвейера

Программный ISP (libcamera soft-ISP или собственный) выполняет последовательность: black level → линейаризация → баланс белого → демозаик (Bayer→RGB) → цветовая коррекция (CCM) → денойз → тонмаппинг/гамма → шарп. Каждая стадия имеет параметры, зависящие от сенсора. Качество результата критически зависит от корректности этих параметров (тюнинга).

17.2 · Цветовая наука и тюнинг

Цвет «как Pixel» определяется матрицами цветокоррекции (CCM), профилем баланса белого по освещённости, профилем шума и кривыми тонмаппинга — всё под конкретный сенсор.

Сообщество (postmarketOS) создаёт такие тюнинг-файлы вручную; на Qualcomm цветокоррекция в libcamera «на подходе» и убирает зелёный оттенок, приближая к Android. Это статус С НУЛЯ и главный потолок качества.

17.3 · Raw / DNG

HDR+ стартует с Bayer-raw, обходя нежелательную обработку аппаратного ISP (тонмаппинг, денойз), что даёт больше бит на пиксель и контроль. Промежуточные кадры удобно хранить/отлаживать как DNG; открытые реализации HDR+ работают именно с бурстами DNG, что позволяет отлаживать весь конвейер на десктопе до появления capture-драйвера.

17.4 · Выравнивание и слияние бурста (HDR+)

Ядро HDR+: захватывается бурст кадров постоянной (низкой) экспозиции (так выравнивание устойчивее и не выбиваются света), кадры выравниваются (FFT-based align) и сливаются (гибридный 2D/3D Wiener-фильтр) для подавления шума, затем применяется HDR-тонмаппинг к чистому изображению с высокой битностью. Открытые реализации (hdr-plus на Halide, hdrplus-python) воспроизводят это ядро; finishing (цвет/контраст) проще и допускает упрощения.

17.5 · Видео-путь

Видео — отдельная и более тяжёлая задача: нужен устойчивый поток с ISP и аппаратный энкодер. На открытых стеках видео обычно отстаёт (на части устройств работает с компромиссами). В нашем проекте видео — поздняя веха с компромиссами по качеству/частоте кадров, зависящая от доступа к энкодеру.

Среда разработки, сборка и CI

Конкретика запуска: машина, сборка Fuchsia, FEMU, тестовые контуры и управление форком.

18.1 · Машина и зависимости

Сборка Fuchsia тяжёлая: нужна машина с многоядерным CPU, значительным объёмом RAM и быстрым диском (десятки гигабайт под дерево и артефакты). Официально поддерживается x86-64 Linux/Mac (по требованиям build-скрипта; не-Debian дистрибутивы могут быть не поддержаны напрямую). Для проекта целесообразна выделенная сборочная машина/CI-раннер.

18.2 · Сборка Fuchsia (GN/ninja/fx)

Дерево собирается через GN (генерация) + ninja (сборка), обернутые в инструменты fx. Типовой цикл: настроить конфигурацию продукта/борда (fx set), собрать (fx build), запустить (эмулятор или железо). Наш форк добавляет свой product/board (substrate) и продуктовые компоненты, собираемые в рамках того же дерева.

18.3 · FEMU и отладка

FEMU (на базе QEMU/AEMU) — основной полигон Трека А. Запуск продукта, итерация UI/агентов, отладка через ffx и GDB (Zircon отлаживается в QEMU; есть публичные разборы практики). Логи — через archivist; трассировка — штатными средствами Fuchsia. Mock-substrate подменяет железо в FEMU.

18.4 · CI и тестовые контуры

CI прогоняет юнит- и интеграционные тесты в FEMU на каждый коммит (логика агентов, хранилище, синк, FIDL-контракты с mock-substrate, Starnix-совместимость). Железо-зависимые тесты (драйверы, bring-up) — отдельный контур на стендах с реальным Pixel 9. Разделение контуров отражает развязку треков.

18.5 · Управление форком и upstream

Форк пиннит конкретную ревизию upstream Fuchsia (защита от движущейся мишени) с периодическими управляемыми ребейзами/мерджами. Нестабильные API (Starnix, новые DFv2-фичи) изолируются за внутренними абстракциями, чтобы апдейты upstream не ломали продукт. Происхождение RE-знаний (например, командные форматы из Panfrost) документируется для прозрачности.

Уроки прецедентов

Что говорят опыт Asahi, Replicant, dahliaOS, postmarketOS и Genode о реалистичности и тактике.

19.1 · Asahi Linux

Asahi (реверс Apple Silicon под Linux) — эталон «маленькая команда реверсит современное железо». Уроки: даже с блестящей командой и годами это огромный труд; ключевым множителем была цель — Linux, где драйверная экосистема уже есть. Для нас это предупреждение: целясь в Fuchsia (без готовой экосистемы) на недокументированном Tensor, мы берём задачу тяжелее Asahi — отсюда обязательность развязки треков.

19.2 · Replicant / libsamsung-ipc

Replicant годами реверсил телефонию Samsung и дал libsamsung-ipc — но покрытие ограничено старыми модемами, и до сих пор не охватывает современные. Урок: модем — самый трудоёмкий и «долгоиграющий» блок; реалистично жить на данных/SMS, относясь к голосу и новым модемам как к рискованным вехам. BaseMirror (авто-RE команд из RIL) — современный множитель, снижающий ручной труд.

19.3 · dahliaOS

dahliaOS — существующий форк Fuchsia (умеет на Zircon и на mainline Linux), пусть и экспериментальный. Урок: форк Fuchsia осуществим и кем-то уже сделан; ценность не в акте форка, а в том, что на нём строится. Подтверждает выбранную стратегию: форк — лёгкая часть, тяжёлая — substrate и продукт.

19.4 · postmarketOS / Megapixels

postmarketOS довёл камеры на Pixel 3A/Fairphone 5 до рабочего состояния через libcamera+Megapixels, честно отмечая «ретро-эстетику» результата и ручную работу над тюнинг-файлами. Урок: открытая вычислительная фотография реальна и переносима, но потолок — тюнинг сенсора, а не алгоритм; ожидания по камере надо калибровать на «достойно», не «Pixel-grade».

19.5 · Genode / Sculpt

Genode/Sculpt показывает, что микроядерная капабилити-ОС может быть ежедневной системой на телефонном железе (PinePhone) — но на открытом железе, где драйверы есть. Урок: микроядро на телефоне достижимо, если снять драйверную стену выбором открытого железа; на закрытом флагмане эта стена остаётся главным барьером.

19.6 · Синтез уроков

- Развязывай продукт и железо — иначе продукт становится заложником многолетнего bring-up.
- Целься в Linux-экосистему там, где можно (Starnix-обёртки) — это резко снижает объём.
- Камеру и голос модема считай рискованными вехами с допустимым провалом, не базой.
- Переиспользуй RE-знания сообщества (Panfrost, libsamsung-ipc, BaseMirror) — не реверсь с нуля без нужды.
- Начинай bring-up с раннего UART и, по возможности, с поддерживаемой платы, не со «слепого» флагмана.
- Калибруй ожидания по качеству: открытые наработки дают фундамент, не фирменную полировку.

Приложения

Полная инвентаризация, расширенный реестр рисков, ссылки, открытые вопросы, глоссарий статусов.

20.1 · А. Полная инвентаризация модулей

Слой	Модуль	Статус
L0	Дерево Fuchsia	ФОРК
L0	Zircon	КАК ЕСТЬ
L0	DFv2	КАК ЕСТЬ
L1	Загрузчик/ZBI	РЕВЕРС
L1	Ядро SoC	РЕВЕРС
L1	Дисплей	РЕВЕРС
L1	Тач/сенсоры	ПОРТ
L1	USB/PMIC/зарядка	РЕВЕРС
L1	UFS	РЕВЕРС/ПОРТ
L1	Wi-Fi/BT	ПОРТ
L1	Titan M2/verified boot	КАК ЕСТЬ
L2	Magma	КАК ЕСТЬ
L2	Mali-драйвер	ПОРТ
L2	GPU firmware	КАК ЕСТЬ
L3	Прошивка модема	КАК ЕСТЬ
L3	Link-транспорт SIPC	ПОРТ
L3	Boot-секвенс	РЕВЕРС
L3	libsamsung-ipc	ПОРТ
L3	Команды Exynos 5400	РЕВЕРС
L3	RIL/ModemManager	ОБЕРНУТЬ
L3	Голос	РЕВЕРС
L4	ISP capture-драйвер	РЕВЕРС
L4	libcamera	ОБЕРНУТЬ
L4	HDR+/Halide	КАК ЕСТЬ
L4	Тюнинг сенсора	С НУЛЯ
L5	Starnix	КАК ЕСТЬ
L6	Shell/композитор	С НУЛЯ
L6	Entity/Agent	С НУЛЯ
L6	Глобальная история	С НУЛЯ

Слой	Модуль	Статус
L6	Данные+синк	С НУЛЯ
L6	Интеграции/виджеты	С НУЛЯ
L6	Apple-интероп	ОБЕРНУТЬ

20.2 · В. Расширенный реестр рисков

#	Риск	Слой	Митигация
R1	Загрузчик/boot-путь частично закрыт	L1	ждать/искать загрузчик; начать с поддержанной платы
R2	SoC bring-up затягивается на годы	L1	ранний UART; переиспользовать downstream-знания
R3	GPU-драйвер не достигает Vulkan	L2	программный рендер; поэтапность; Panfrost-знания
R4	Голос модема не заводится	L3	допустить провал вехи; жить на данных/SMS
R5	Команды Exynos 5400 неполны	L3	BaseMirror + наблюдение; итеративно
R6	ISP/тюнинг не дотягивает	L4	принять postmarketOS-уровень качества
R7	Starnix-ABI ломает компоненты	L5	пиннинг ревизии; тесты совместимости
R8	Движущиеся API Fuchsia	все	пиннинг + абстракции над нестабильным
R9	Объём недооценён/выгорание	все	развязка треков; MVP; принятие урезаний

20.3 · С. Ключевые проекты и ссылки

Что	Ссылка
Fuchsia / Zircon / DFv2 / Magma	https://fuchsia.dev
Starnix	https://fuchsia.dev/fuchsia-src/concepts/starnix/kernel
dahliaOS (форк Fuchsia)	https://github.com/dahliaOS/fuchsia
Panfrost/Panthor/PanVK/Tyr (Mesa)	https://gitlab.freedesktop.org/ mesa/ mesa
hdr-plus (Halide)	https://github.com/timothybrooks/hdr-plus
hdrplus-python (IPOL)	https://github.com/amonod/hdrplus-python
hdr-plus-pytorch	https://github.com/martin-marek/hdr-plus-pytorch
HDR+ dataset	http://www.hdrplusdata.org/
Halide	https://halide-lang.org
libcamera	https://libcamera.org
Megapixels (postmarketOS)	https://gitlab.com/megapixels-org/Megapixels
Millipixels (Librem 5)	https://source.puri.sm/Librem5/millipixels
libsamsung-ipc (Replicant)	https://redmine.replicant.us/projects/replicant/wiki/Libsamsung-ipc
BaseMirror (RE команд Exynos RIL)	https://arxiv.org/abs/2409.00475
HDR+ (Hasinoff et al., 2016)	https://research.google/pubs/burst-photography-for-high-dynamic-range-and-low-light-imaging-on-mobile-cameras/
postmarketOS: камеры (Pixel 3A/FP5)	https://postmarketos.org/blog/2024/12/23/v24.12-release/

20.4 · D. Открытые вопросы

- Доступен ли путь загрузки Zircon на Pixel 9 без «невыпущенного» загрузчика, упомянутого в патчах для Pixel 10?
- Насколько BaseMirror покрывает командный набор именно Exynos 5400 (а не более старых Exynos)?
- Реально ли завести голосовой тракт без проприетарного аудио-HAL — или это потолок телефонии?
- Какой объём тюнинга сенсора нужен для приемлемого фото, и переносим ли тюнинг между ревизиями?
- Стабильность Starnix-ABI на горизонте проекта (учитывая, что это движущаяся мишень Google).
- Стоит ли вместо Pixel 9 рассмотреть промежуточную ARM-плату с готовой поддержкой Fuchsia для отработки substrate.

20.5 · E. Легенда статусов (справочно)

Статус	Значение
КАК ЕСТЬ	берём готовое без изменений
ОБЕРНУТЬ	запускаем существующее в обёртке (Starnix)
ПОРТ	переносим код/знания на нашу платформу
РЕВЕРС	реверс-инжиниринг с нуля
ФОРК	форкаем и ведём свою ветку
С НУЛЯ	пишем новый код